

SkillGraph

Explainable Implicit Skill Inference & Resume Evaluation System

Stack: Python 3.10+ | Flask | spaCy | SBERT | NetworkX | React 19 | Vite

Architecture: 7 Pipeline Stages
(Offline / Zero-API)

Abstract & Problem Statement: Candidates rarely enumerate every foundational tool, protocol, or concept they use, yet standard keyword-matching Applicant Tracking Systems (ATS) automatically disqualify them for unstated implicit competencies. Conversely, generative LLMs often hallucinate unverified skills and fabricate accomplishments. **SkillGraph** bridges this gap through deterministic skill gazetteers, directed graph knowledge traversal, contextual Sentence-BERT similarity, and active candidate verification to ensure explainable, zero-hallucination resume evaluations.

1. Technology Stack: Backend & Frontend

Component Layer	Technologies & Libraries	Role in System
Backend Runtime	Python 3.10+, Flask 3.x	Core pipeline orchestration & REST API endpoints
Linguistic Parsing	spaCy (en_core_web_sm)	Bullet/sentence segmentation & entity cross-checking
Semantic Embeddings	Sentence-BERT (all-MiniLM-L6-v2)	384-d dense embeddings; offline cosine similarity (CPU)
Knowledge Graph	NetworkX (DiGraph)	68 canonical skills, 64 directed weighted edges
Document Ingestion	pdfplumber, python-docx	Stream parsing, layout repair, text extraction
Backend Security	Flask-Limiter, Magic Bytes, Decompress Bomb Defense	Rate limits (10/min), file type validation, 5 MB ceiling
Frontend Framework	React 19, Vite 8	Reactive 3-stage UI, rapid HMR, optimized bundle
Styling & Typography	Tailwind CSS v4, Instrument Serif, Plus Jakarta, JetBrains Mono	Academic institutional prestige theme, print stylesheets
Frontend Interaction	Native Clipboard & Print APIs, Keyboard Shortcuts	Hotkeys [1][2][3] for Q&A;., instant 1-click sample loader

2. End-to-End System Flow & Architecture

SkillGraph operates across 5 linear pipeline execution stages triggered via the RESTful server and rendered dynamically in the React client:

STAGE 1: Ingestion & Segmentation <i>File & Document Parsing</i> (skillgraph/parser.py)	- Validates magic bytes (%PDF-, PK\x03\x04) and guards against zip bombs. - Normalizes carriage returns, cleans OCR artifacts, rejoins line wraps. - Uses spaCy to segment text into discrete, actionable statements.
STAGE 2: Explicit Extraction <i>Gazetteer Matching</i> (skillgraph/extractor.py)	- Matches resume & job description against 68 canonical skills and 300+ aliases. - Longest-match greedy regex with word boundaries to eliminate substrings. - Records precise statement-level provenance for every matched skill.
STAGE 3: Inference Engine <i>Graph & SBERT</i> (skillgraph/inference.py)	- Identifies missing skills required by the job description. - Checks NetworkX graph for prerequisite edges from candidate's skills. - Computes local SBERT embedding cosine similarity against source statement. Formula: Confidence = (0.50 * w_edge + 0.50 * sim_SBERT) * 100 - Tri-tier classification: High (≥85%), Needs Verification (40-85%), Gap (<40%).
STAGE 4: Active Verification <i>Candidate Disambiguation</i> (skillgraph/verifier.py)	- Generates deterministic questions exclusively for NEEDS_VERIFICATION. - Candidate responds: <i>Hands-on</i> (1.0), <i>Framework/Library</i> (0.5), or <i>Unfamiliar</i> (0.0). - Eliminates false positives before report synthesis.
STAGE 5: Report & Rewriting <i>Synthesis & Slots</i> (skillgraph/reporter.py)	- Calculates weighted overall fit percentage. - Deterministic slot-filling rewrites augment resume statements without fabricating metrics or unverified tools. - Generates comprehensive executive dossier with full algebraic explanations.
STAGE 6: Interactive Dashboard <i>Web Interface & API</i> (server.py + frontend/)	- Flask session-isolated REST backend with in-memory TTL caching. - React 19 UI with real-time fit gauges, keyboard shortcuts, and print stylesheet.

3. Mathematical Formulations & Algorithms

Confidence Calculation (Stage 3):

For each required skill s absent from the candidate's explicit skills, SkillGraph finds all predecessors $p \in \textit{Explicit}$ in the directed skill graph where $(p \rightarrow s)$ exists with edge weight $w(p, s)$. It then measures the contextual semantic similarity between s and the source statement $stmt(p)$ using Sentence-BERT:

$$\text{Confidence}(s) = [0.50 \times w(p, s) + 0.50 \times \text{CosineSimilarity}(\text{SBERT}(s), \text{SBERT}(stmt(p)))] \times 100$$

Candidate Fit Percentage (Stage 5):

The overall candidate qualification score accounts for explicit proficiencies, confirmed implicit proficiencies, and partial library-level exposures, normalized over the total skills demanded by the target job description:

$$\text{Fit \%} = \text{round}([N_{\text{explicit}} \times 1.0 + N_{\text{confirmed}} \times 1.0 + N_{\text{high_unverified}} \times 0.85 + N_{\text{partial}} \times 0.5] / N_{\text{total_required}} \times 100)$$

4. Stage 7 Empirical Evaluation Benchmark

The inference engine was evaluated against a rigorous ground-truth dataset (eval_dataset.json) comprising 18 hand-labeled resume/job-description scenarios across 60 evaluated skill targets:

Metric	Measured Score	Benchmark Significance & Interpretation
Inference Precision	94.1% (16 / 17)	16 of 17 inferred skills were genuine, justifiable competencies.
Inference Recall	94.1% (16 / 17)	Successfully recovered 16 of 17 unstated skills expected by evaluators.
Inference F1-Score	94.1%	Harmonic balance of precision and recall under zero-shot testing.
False Positive Rate on Gaps	5.0% (1 / 20)	Only 1 genuine skill deficiency out of 20 was incorrectly flagged.
Overall Accuracy	96.7% (58 / 60)	Exact classification match across all explicit, inferred, and gap skills.

5. RESTful API Interface Specification

HTTP Method & URI	Request Payload	Function & Returned Data
GET /api/health	None	Health check returning operational status and uptime.
POST /api/analyze	Multipart: resume (file) + job_description (text)	Runs Stages 1-3, registers session, returns matched skills & questions.
POST /api/answer	JSON: {session_id, missing_skill, selected_option}	Applies Stage 4 update (CONFIRMED/PARTIAL/GAP) and returns next question.
GET /api/report/<id>	URL parameter: session_id	Synthesizes Stage 5 final score, skill breakdowns, and bullet rewrites.

6. Core Architectural Guarantees

- **Local Privacy:** All tensor calculations and NLP operations run strictly on-device without telemetry or cloud calls.
- **Full Explainability:** Every recommendation is grounded in an exact resume sentence, a knowledge graph edge, and a mathematical similarity score.
- **Zero Hallucination:** Resume rewrites use strict slot-filling templates anchored to original statements, never fabricating metrics, companies, or tools.
- **Human-in-the-Loop Disambiguation:** Ambiguous skills (40% - 85% confidence) require candidate confirmation, capping the false positive rate to 5.0%.